Receiver-Guided Reinforcement Learning-based Congestion Control for IoT

Qihang Jiao, *Student Member, IEEE,* and Lotfi Mhamdi, *Senior Member, IEEE*

***Abstract*—The abstract goes here.**

***Index Terms*—IoT, TCP, congestion control, reinforcement learning.**

I. Introduction

THIS part is to be completed.

II. Related Works

III. System Design

Our proposal contains three modules in the receiver side for adjusting sender $cwnd$, Receiver Measurement Module, Window Decision Module, and Window Enforcement Module. Additionaly, TCP sockets in both sender and receiver are extended or modified for adding interfaces to achieved our features.

A. Control Message types definition

The receiver TCP socket is extended to support communication with the RL agent. Three types of messages are defined:

- **State**: carries the observed network state and is sent to the agent for storage in the replay buffer.
- **Action Request**: triggers the agent to infer and return a control action based on the current state.
- **Connection Close**: notifies the agent that a flow has terminated, allowing it to clear the corresponding internal states.

B. Receiver Measurement Module

Our scheme relies on reliable state observations to ensure stable training. Therefore, the states in our design are required to reflect the congestion extent while remaining observable at the receiver. Despite RTT is a fine-grained and valuable indicator of congestion, it is difficult to directly observe in the receiver side.

To address this limitation, we derive a receiver-side congestion entent metric based on the $cwnd$ elapsed time. We extend the sender TCP feature to include its $cwnd$ in TCP header. In standard TCP congestion avoidance, the congestion window $cwnd$ is increased by 1 MSS after an entire $cwnd$ bytes has been acknowledged, resulting in an approximate growth rate of 1 MSS per RTT, in which the method is named Additive Increment.

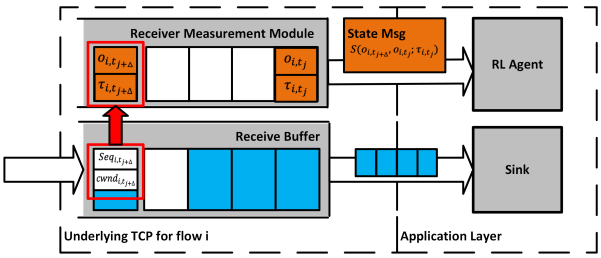

Fig. 1. An example of estimation task completion of flow i.

We utilise Additive Increment growing feature to enable the receiver to estimate network congestion extent by calculating the elapsed time for a flow to complete its $cwnd$. In particular, upon every packet receive event that includes a $cwnd$ value different from the last value received, the Receiver Measurement Module establishes a new estimation task.

A new task is initialised with a record for the current flow state and a threshold (τ), e.g., for flow i started at time t_0 and estimated at time t_j, defined as below:

$$record_{i,t_j} \triangleq [B_i(t_j), L_i(t_j), t_j] \tag{1}$$

$$\tau_{i,t_j} = Seq_{i,t_j} + cwnd_{i,t_j} \tag{2}$$

$$B_i(t_j) = \sum_{n=t_0}^{t_j} RxBytes_{i,n} \tag{3}$$

$$L_i(t_j) = \sum_{n=t_0}^{t_j} Losses_{i,n} \tag{4}$$

An estimation task is completed at $t_{j+\Delta}$ when a coming packet is seen with a sequence number satisfying $Seq_{i,t_{j+\Delta}} > \tau_{i,t_j}$. We calculate this task duration as the $cwnd$ elapsed time i.e., $t_{elapsed} = t_{j+\Delta} - t_j$. We then calculate the goodput (g_{i,t_j}) and packet losses during the estimation task, where $g_{i,t_j} = \frac{B_i(t_{j+\Delta}) - B_i(t_j)}{t_{elapsed}}$. and $\ell_{i,t_j} = L_i(t_{j+\Delta}) - L_i(t_j)$. We then form an observation according to $o_{i,t_j} = O(record_{i,t_{j+\Delta}}, record_{i,t_j}; \tau_{i,t_j})$, where the generation process is defined as follows:

$$O(record_{i,t_{j+\Delta}}, record_{i,t_j}; \tau_{i,t_j}) \triangleq \left[g_{i,t_j}, \ell_{i,t_j}, t_{elapsed}\right]$$

The Receiver Measurement and State Generation process can be seen in Algorithm 1.

Qihang Jiao and Lotfi Mhamdi are with the School of Electronic and Electrical Engineering, University of Leeds, Leeds, UK (e-mail: el22qj@leeds.ac.uk; L.Mhamdi@leeds.ac.uk).

Lotfi Mhamdi is also with the College of Computing & Systems, Abdullah Al Salem University, Kuwait (e-mail: lotfi.mhamdi@aasu.edu.kw).

Algorithm 1 Receiver Measurement and State Generation

Input: Flow Id i, Time t_j, Arrived Packet p_{i,t_j}, Flow Observation Queue q_i
Output: Observation o_{i,t_j}, Control Message M

Extract Seq_{i,t_j}, $cwnd_{i,t_j}$, and $RxBytes_{i,t_j}$ from p_{i,t_j}
$\tau_{i,t_j} = Seq_{i,t_j} + cwnd_{i,t_j}$
Obtain $Losses_{i,t_j}$ from $TcpSocket_i$
$B_i(t_j) = B_i(t_{j-1}) + RxBytes_{i,t_j}$
$L_i(t_j) = L_i(t_{j-1}) + Losses_{i,t_j}$
$record_{i,t_j} = [B_i(t_j), L_i(t_j), t_j]$
Obtain $record_i^{head}$ and τ_i^{head} from the head of q_i
if $Seq_{i,t_j} \geq \tau_i^{head}$ **then**
    Dequeue $record_i^{head}$ and τ_i^{head} from q_i
    $o_{i,t_j} = O(record_{i,t_j}, record_i^{head})$
    Include o_{i,t_j} into M
    Send M to agent
else if $cwnd_{i,t_j} \neq cwnd_{i,t_{j-1}}$ **then**
    Enqueue $record_{i,t_j}$ and τ_{i,t_j} into q_i
end if

Algorithm 2 Window Decision Module

Input: Control Message M, Replay Buffer B, Flow State Buffer B_i for flow i, Batch Size N
Output:

if M = "Connnection Close" **then**
    Clear B_i
else if M = "State" **then**
    Extract s_i from M
    Store s_i into B_i
else if M = "Action Request" **then**
    Compute the average state $\bar{s_i}$ over B_i
    $a = \mu(\bar{s_i}; \theta^\mu)$
    Store transition $(\bar{s_i}, a_i, r_i, \bar{s_i}')$ into B
    Randomly sample N transition from B
    Compute $y_t = r(s_t, a_t) + \gamma Q^-(s_{t+1}, \mu^-(s_{t+1}))$
    Compute $L(\theta^Q) = \mathbb{E}[(Q(s_t, a_t; \theta^Q) - y_t)^2]$
    Update Critic minimising $L(\theta^Q)$
    Compute $\nabla_{\theta^\mu} J(\theta^\mu) = \mathbb{E}[\nabla_{a_t} Q(s_t, \mu(s_t)) \nabla_{\theta^\mu} \mu(s_t)]$
    Update Actor via gradient ascent $\nabla_{\theta^\mu} J(\theta^\mu)$
    $\theta^{\mu^-} \leftarrow \tau \times \theta + (1-\tau) \times \theta^{\mu^-}$
    $\theta^{Q^-} \leftarrow \tau \times \theta + (1-\tau) \times \theta^{Q^-}$
end if

C. Window Decision Module

Window Decision Module consists of a Reinforcement Learning model to output a scaling factor to underlying TCP socket and a periodic action request feature. An enforced congestion window is calculated with the factor by the underlying TCP socket and included in ACK to the sender. The sender will overwrite its $cwnd$ by the enforced congestion window. Therefore, to implement a continuous control to modified TCP, we choose Deep Deterministic Policy Gradient (DDPG) as agent in our design.

1) Training: Our agent consists of an actor network determining a deterministic action $a = \mu(s; \theta^\mu)$ parameterised by θ^μ and a critic network to approximate the expected future return, defined as $Q(s, a; \theta^Q) = \mathbb{E}[R_t | s_t, a_t]$ with parameters θ^Q. The return R_t is the sum of discounted future return, i.e., $R_t = \sum_{i=t}^{T} \gamma^{i-t} r(s_i, a_i)$. Approaching to the approximation is done by updating the critic network through minimising the mean square error between its output and a target function (y_t), shown as follows:

$$L(\theta^Q) = \mathbb{E}[(Q(s_t, a_t; \theta^Q) - y_t)^2]$$

$$y_t = r(s_t, a_t) + \gamma Q^-(s_{t+1}, \mu^-(s_{t+1}; \theta^{\mu^-}); \theta^{Q^-})$$

where $Q^-(s, a; \theta^-)$ and $\mu^-(s; \theta^{\mu^-})$ are target critic network and target policy network. Their parameters are updated by calculating the exponetially weighted moving average of critic network and policy network, i.e., $\theta^- \leftarrow \tau \times \theta + (1-\tau) \times \theta^-$ where the τ is the weight parameter. Subsequently, with the updated critic network, policy performance of current actor network is predicted, expressed as $J(\theta^\mu) = \mathbb{E}[Q(s_t, \mu(s_t; \theta^\mu))]$. Training an actor network is done by updating its paramters through policy performance gradient ascent. Specifically, by applying the chain rule, the gradient of the policy performance, is computed as:

$$\nabla_{\theta^\mu} J(\theta^\mu) = \mathbb{E}[\nabla_{a_t} Q(s_t, \mu(s_t; \theta^\mu)) \nabla_{\theta^\mu} \mu(s_t; \theta^\mu)]$$

2) Reward: The reward function is incorporated into the target function defining the direction we update the agent parameters.

D. Window Enforcement Module

For an RL-controlled CC, it is important that waiting agent for action generation should not block the TCP normal behaviour. Therefore, we decouple the Window Decision Module, Receiver Measurement Module and standard feature to ensure none of them will block any other. Therefore, an ACK response will not be delayed due to our scheme. In our design, an action request message will be sent to agent periodically by Window Decision Module. The resulting action is a factor within $[0, 1]$. Once the Window Decision Module receive the action, it will extract the latest $cwnd$ in Receiver Measurement Module and compute the enforced congestion window using the following:

$$cwnd_{enforced} = cwnd \times scaling\_factor^a, \ n \in [0, 1]$$

Then the enforced $cwnd$ will be included into the ACK header and take effect at the sender side. For the receiver, the same action will keep effective untill the next action request.

IV. Simulation setup and results

V. Conclusion

The conclusion goes here.

Acknowledgment

The authors would like to thank...

References

[1] H. Kopka and P. W. Daly, *A Guide to LaTeX*, 3rd ed. Harlow, England: Addison-Wesley, 1999.